

BANKMANAGEMENT

System Security, Authentication, and Authorization

Technical Design and Implementation Reference

Microsoft SQL Server | Node.js / Express | Vanilla JavaScript

Version 1.0 | July 2026

Document purpose

This document describes the security controls implemented in the BankManagement application, with particular emphasis on identity, login, session validation, role evaluation, branch and ownership scope, database permissions, and auditability. It distinguishes application users from SQL Server connection identities and explains where each security decision is enforced.

Core security principle

The frontend is not treated as a security boundary. A request must pass session validation, API authorization, scope checks, and stored-procedure business rules before protected data is returned or changed.

Contents

1. Security architecture and trust boundaries
 2. Identity and credential model
 3. Authentication lifecycle
 4. Authorization and effective roles
 5. Ownership and branch scope
 6. Database access and SQL injection protection
 7. Database-level least privilege
 8. Auditing, integrity, and operational controls
 9. Error handling and session failure behavior
 10. Current limitations and improvement priorities
- Appendix A. Key implementation files

Executive summary

BankManagement uses a layered security model. The browser holds an opaque session token, the Express API validates that token through SQL Server on every protected request, route middleware checks broad application roles, and stored procedures enforce the final ownership, branch, state, and business constraints. SQL Server service accounts are separated from application identities, and database roles restrict technical users to procedure execution or approved reporting views.

- **Authentication:** Passwords are verified in SQL Server and successful login creates a database-backed session with a fixed expiration time.
- **Authorization:** Raw role assignments are converted into effective roles based on the current user, customer, employee, job title, admin flag, and employment state.
- **Scope enforcement:** Customers are limited to their own records; employees and managers are generally limited to their current branch; HighAdmin has central scope.
- **Data access:** The backend binds typed parameters and executes stored procedures instead of concatenating user values into SQL text.
- **Traceability:** Login, logout, session expiry, administrative actions, searches, and major business operations are written to AuditLog.

1. Security architecture and trust boundaries

The system separates four distinct security contexts. Keeping these contexts separate prevents a browser user from inheriting the SQL Server privileges of the backend process.

Layer	Identity or control	Primary responsibility
Browser	Session token and displayed user context	Collect credentials, attach the bearer token, and render only the permitted user interface.
Express API	req.user reconstructed from session validation	Authenticate requests, apply route-level role checks, determine requested scope, and bind typed database parameters.
Application database model	dbo.Users, dbo.UserRoles, dbo.Customer, dbo.Employee, dbo.EMPB	Calculate effective roles and enforce customer ownership, branch correspondence, employment status, and

Layer	Identity or control	Primary responsibility
		business rules.
SQL Server technical security	Database logins, users, roles, grants, and controlled views	Limit what the backend, reporting, audit, and maintenance connections can execute or read.

1.1 Request trust chain



2. Identity and credential model

2.1 Application identity

Application users are stored in dbo.Users. A user is linked to one Customer record and may also be linked to one Employee record. Role assignments are stored in dbo.UserRoles. Unique filtered indexes prevent more than one user account from being linked to the same CustomerID or EmployeeID.

Table	Security purpose	Important fields
dbo.Users	Login identity and account activation	UserID, Username, PasswordHash, CustomerID, EmployeeID, IsActive
dbo.Roles	Defines Customer, Employee, Admin, and HighAdmin	RoleID, RoleName

Table	Security purpose	Important fields
dbo.UserRoles	Many-to-many role assignment with duplicate prevention	UserID, RoleID, UNIQUE(UserID, RoleID)
dbo.Sessions	Persistent server-side login sessions	SessionToken, LoginTime, ExpiresAt, LogoutTime, IsActive
dbo.AuditLog	Trace of security and business actions	UserID, ActionType, TableName, RecordID, ActionDate, Details

2.2 Password storage

The current implementation never stores the plaintext password. Signup, HighAdmin creation, and login use `dbo.fn_HashPassword`, which converts the password to an uppercase SHA-256 hexadecimal value stored in `Users.PasswordHash` as `CHAR(64)`.

Implemented password-hash function

```
CREATE FUNCTION dbo.fn_HashPassword(@Password NVARCHAR(4000))
RETURNS CHAR(64)
AS
BEGIN
    RETURN CONVERT(
        CHAR(64),
        UPPER(CONVERT(VARCHAR(64), HASHBYTES('SHA2_256', @Password), 2))
    );
END;
```

Customer signup requires a password of at least eight characters. The initial HighAdmin bootstrap procedure currently requires at least six characters, which should be aligned with the stronger rule.

Important security limitation

SHA-256 is a fast general-purpose hash and the current function does not use a per-user salt. It prevents plaintext storage but is not suitable as the final password-storage design for a production banking application. An adaptive password hash such as Argon2id, bcrypt, or PBKDF2 with a unique salt should replace it.

2.3 Application users versus SQL Server users

A Customer, Employee, Admin, or HighAdmin is an application identity in `dbo.Users`. BankAppLogin, BankReportLogin, and the other SQL Server logins are technical identities used by backend connection pools. A customer never connects directly to SQL Server and never receives a SQL Server login.

3. Authentication lifecycle

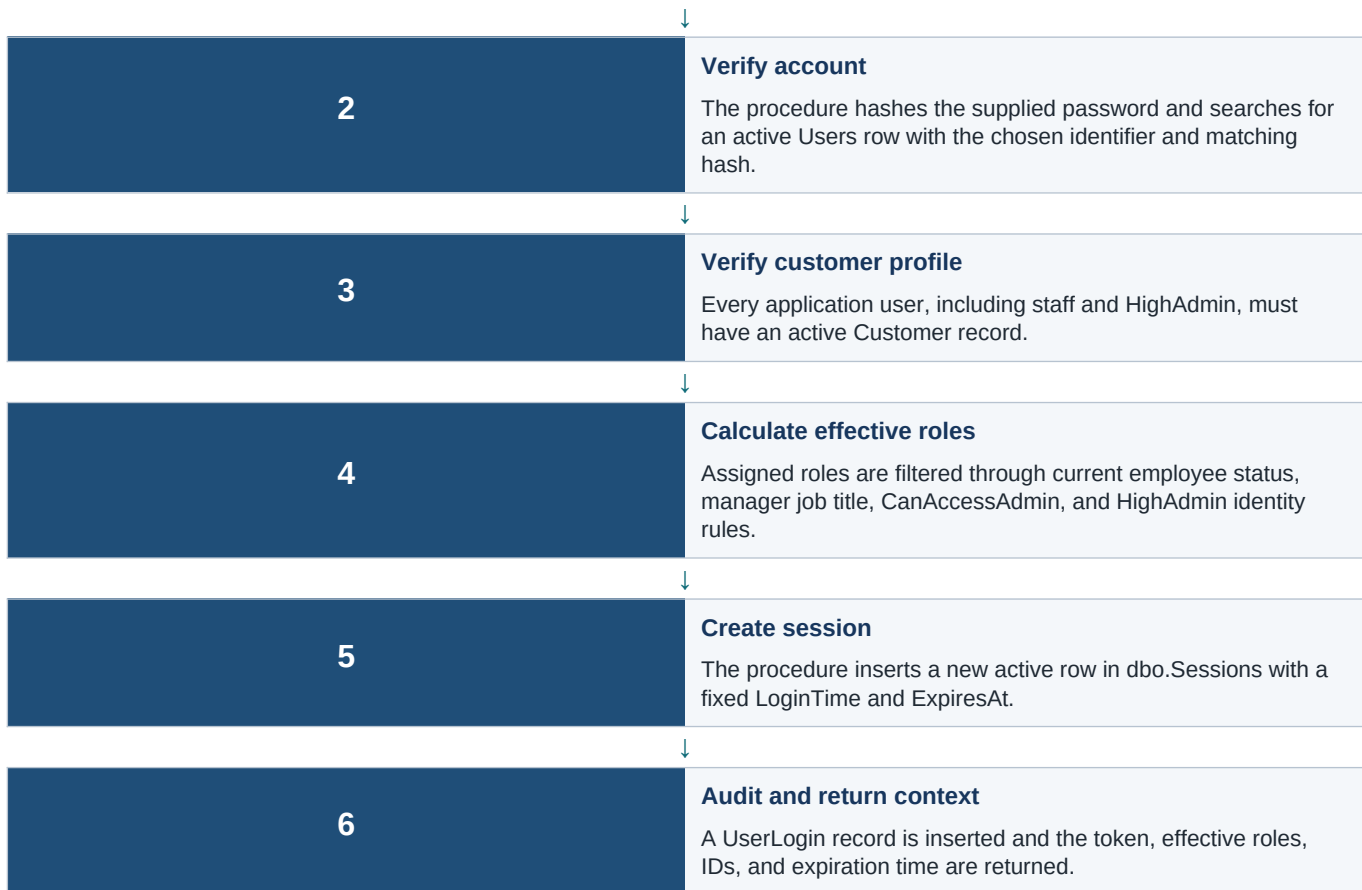
3.1 Signup

1. POST `/api/auth/signup` requires username, password, personal information, and contact information.
2. The API binds those values to `dbo.sp_User_SignUp` using explicit SQL Server data types.
3. The procedure validates username length, password length, and username uniqueness.
4. A transaction creates the Customer, creates an active Users row containing only the password hash, grants the Customer role, and writes a UserSignUp audit record.
5. If any step fails, `XACT_ABORT` and the TRY/CATCH transaction logic roll back the complete signup operation.

3.2 Login

The login route is protected by an express-rate-limit policy of 20 attempts per 15-minute window. The procedure accepts exactly one login identifier - Username, UserID, CustomerID, or EmployeeID - together with the password. Numeric IDs are treated only as identifiers and do not replace the password.

1	Validate input Exactly one identifier and a non-empty password are required. Session lifetime is restricted to 1-1440 minutes.
---	--



3.3 Session token and expiration

The session token is opaque rather than a JWT. It is produced from two GUID text values separated by a hyphen, resulting in a 73-character token. The token is stored in dbo.Sessions and sent by the client in the Authorization header.

Session creation excerpt

```
SET @SessionToken = CONVERT(NVARCHAR(36), NEWID())
                    + N'-'
                    + CONVERT(NVARCHAR(36), NEWID());

INSERT INTO dbo.Sessions
  (UserID, SessionToken, LoginTime, ExpiresAt, LogoutTime, IsActive)
VALUES
  (@UserID, @SessionToken, @LoginTime, @ExpiresAt, NULL, 1);
```

- **Default lifetime:** 10 minutes, configurable through SESSION_TTL_MINUTES and constrained to 1-1440 minutes.
- **Expiration model:** Fixed absolute expiration. Activity does not extend ExpiresAt.
- **Multiple sessions:** Login creates a new session and does not automatically revoke other unexpired sessions for the same user.
- **Storage in the browser:** The frontend uses localStorage when persistence is selected and sessionStorage otherwise.
- **Storage in SQL Server:** SessionToken is unique; an index on IsActive and ExpiresAt supports expiration checks.

3.4 Protected request validation

Simplified authentication middleware

```
function authenticate(requiredRole = null) {
  return async function authMiddleware(req, res, next) {
    const token = extractBearerToken(req);
    if (!token) return res.status(401).json(...);

    const result = await executeProcedure('dbo.sp_User_ValidateSession', {
      inputs: { SessionToken: TYPES.token, RequiredRole: TYPES.string50 },
      values: { SessionToken: token, RequiredRole: requiredRole }
    });

    req.sessionToken = token;
    req.user = { ...result.recordset[0], roles: parseRoles(result.recordset[0].EffectiveRoles) };
    return next();
  };
}
```

sp_User_ValidateSession verifies that the token exists, the session is active, ExpiresAt is in the future, the user is active, and the linked customer is active. It then recalculates effective roles and current branch information from current database state. This means a role can disappear immediately on the next request after an employee is suspended, terminated, downgraded, or loses admin eligibility.

State-sensitive authorization

An OnLeave or Terminated staff member may retain a valid session as Customer, but Employee and Admin privileges are removed during session validation. The session context is not trusted from the original login response.

3.5 Logout and expired sessions

POST /api/auth/logout first validates the bearer token, then dbo.sp_User_Logout locks the active session row, sets IsActive to 0, records LogoutTime, and writes UserLogout to AuditLog. When validation detects an expired session, the procedure deactivates it, records SessionExpired, and rejects the request with an authentication error.

4. Authorization and effective roles

4.1 Assigned roles versus effective roles

UserRoles records what was assigned historically. Effective roles represent what the user is allowed to use now. Both login and session validation calculate effective roles from current data instead of returning raw role rows unchanged.

Role	Conditions for effectiveness
Customer	Active Users row, active linked Customer row, and assigned Customer role.
Employee	Customer conditions plus linked Employee, assigned Employee role, and Employee.EmpStatus = Active.
Admin	Effective Employee plus assigned Admin role, CanAccessAdmin = 1, and JobTitle is Branch Manager or Vice Manager.
HighAdmin	Active customer-linked user with assigned HighAdmin role and no EmployeeID. HighAdmin must not be combined with Employee or Admin roles.

4.2 API role middleware

Route-level role check

```
function authorize(...allowedRoles) {
  return function roleMiddleware(req, res, next) {
    const roles = req.user?.roles || [];
    const allowed = allowedRoles.some((role) => roles.includes(role));
    if (!allowed) return res.status(403).json(...);
    return next();
  };
}
```

Route middleware is used for broad operation categories. Examples include Admin or HighAdmin for account unfreezing and maintenance tasks, Admin for ordinary employee hiring, and HighAdmin for central manager operations. Routes without an `authorize(...)` call still pass `authenticate()` and rely on ownership or stored-procedure checks.

4.3 General role access model

Capability	Customer	Employee	Admin	HighAdmin
Personal profile, accounts, loans	Own	Own	Own	Own
Customer/account directory	No general directory	Current branch	Current branch	All branches
Financial transactions	Own source/destination account	Own personal accounts only	Own personal accounts only	Own personal accounts only
Account administrative actions	No	Branch-scoped where allowed	Branch-scoped plus elevated actions	All branches
Employee operations	No	Read within scope	Manage allowed staff in branch	Central manager and cross-branch oversight
Maintenance and reporting	Personal only	Operational branch reports	Branch management and maintenance	Central reports, audit, and maintenance

The table summarizes the intended scope. The exact decision remains procedure-specific because each operation also checks status, ownership, branch assignment, and other business constraints.

5. Ownership and branch scope

5.1 Ownership scope

Financial transaction routes call `assertAccountOwner` before executing deposit, withdrawal, or transfer procedures. The helper forces an account search with `CustomerID` taken from `req.user`, not from the request body. The corresponding transaction procedures also receive the server-derived `UserID` and recheck ownership in SQL Server.

Trusted identity is assigned after client input

```
values: {
  ...req.body,
  EmployeeID: req.user.EmployeeID || null,
  UserID: req.user.UserID
}
```

Because `UserID` is assigned after spreading `req.body`, a client-supplied `UserID` cannot override the authenticated identity. The same pattern is used throughout protected routes.

5.2 Branch scope

For an effective Employee or Admin, the current branch is derived from the latest EMPB row where `WorkingStatus = Working` and `EndDate IS NULL`. The browser cannot choose this branch to enlarge its access. Customer-to-branch correspondence is derived through Account because Customer does not contain a `BranchID` column.

- **Customer:** Scope resolves to the authenticated `CustomerID`.
- **Employee/Admin:** Scope resolves to `CurrentBranchID` from the active EMPB assignment.
- **HighAdmin:** Scope resolves to all branches unless an operation applies a more specific business rule.

5.3 Defense in depth inside stored procedures

The API role check is intentionally not the final authorization decision. Procedures such as `sp_Customer_Search` call `dbo.fn_UserHasEffectiveRole`, resolve the requester identity again, determine the active branch, and apply branch filters in the `SELECT` statement. Direct execution with an unauthorized `@UserID` is rejected even if the API layer were bypassed.

Why the procedure receives UserID

The SQL procedure needs the application identity so it can enforce business authorization using database state. The SQL connection login identifies the backend service, not the customer or employee who initiated the request.

6. Database access and SQL injection protection

6.1 Typed parameter binding

The backend uses the mssql driver directly. Each request creates a parameterized SQL request and binds values with `request.input(name, type, value)`. User values are not concatenated into stored-procedure calls.

Procedure execution helper

```
request.input(name, type, valueOrNull(source[sourceName]));
const result = await request.execute(procedureName);
```

Typed binding separates SQL code from data. A value containing quotes or SQL keywords remains a value and is not interpreted as part of the SQL command. This protection depends on continuing to avoid string concatenation inside stored procedures and ad hoc queries.

6.2 Controlled dynamic report queries

Report view names, allowed roles, ordering expressions, and database pools are selected from a server-owned `REPORT_VIEWS` object. The client supplies only a report key. Unknown keys are rejected, and paging and branch values are bound as integers. This prevents a client from injecting arbitrary table names or `ORDER BY` expressions.

Identifier whitelist

SQL parameters can represent values but not identifiers such as table names. The report module solves this by mapping public report keys to fixed server-side view and order definitions.

6.3 Transactions, locks, triggers, and constraints

Authentication and authorization controls are complemented by integrity controls. Stored procedures use transactions, `XACT_ABORT`, row locks where needed, and `TRY/CATCH` rollback behavior. Foreign keys, unique constraints, `CHECK` constraints, and triggers prevent invalid state transitions or inconsistent multi-table updates. These controls reduce the impact of concurrency and programming errors after authorization has succeeded.

7. Database-level least privilege

SQL Server security is separated into technical personas. Project roles first have direct `SELECT`, `INSERT`, `UPDATE`, and `DELETE` permissions revoked from `dbo` tables. Permissions are then granted through stored procedures or approved views.

Database role	Mapped user	Granted capability
<code>dbrole_bank_app_executor</code>	<code>BankAppUser</code>	<code>EXECUTE</code> on <code>dbo</code> schema and <code>SELECT</code> on approved operational views.
<code>dbrole_bank_reporting</code>	<code>BankReportUser</code>	<code>SELECT</code> on reporting and summary views only.
<code>dbrole_bank_auditor</code>	<code>BankAuditorUser</code>	<code>SELECT</code> on safe audit, branch ledger, and oversight views.
<code>dbrole_bank_highadmin_viewer</code>	<code>BankHighAdminReportUser</code>	Read-only <code>SELECT</code> on central oversight views.
<code>dbrole_bank_maintenance</code>	<code>BankMaintenanceUser</code>	<code>VIEW DEFINITION</code> by default; DBA work remains separate.

The Node.js backend defines separate app, report, audit, and highadminReport connection pools. In production, each pool should use its matching SQL Server login. The configuration currently falls back to the main DB credentials when optional report credentials are omitted, so separate credentials must be set to obtain true permission isolation.

Least-privilege boundary

Application `HighAdmin` and `BankHighAdminReportUser` are different concepts. The former is a person represented in `dbo.Users`; the latter is a SQL Server technical principal used for a read-only reporting connection.

8. Auditing, integrity, and operational controls

8.1 Audit trail

AuditLog records the acting application UserID when available, an action type, target table and record, timestamp, and details. Current procedures record successful login, logout, expiration, signup, customer searches and views, account actions, employee and manager actions, transaction processing, loan state changes, and automated maintenance operations.

- **Security events:** UserLogin, UserLogout, SessionExpired, HighAdminCreated, user activation changes.
- **Read events:** Selected directory searches and profile lookups include scope information.
- **Business events:** Transaction, account, loan, employee, transfer, and interest operations record target IDs and context.
- **System events:** Automated Agent jobs may use UserID = NULL to distinguish system activity from an interactive user.

sp_User_ValidateSession intentionally does not write an audit row for every request because that would create excessive audit volume. Login, logout, expiration, and protected business procedures provide the meaningful trace points.

8.2 HTTP and API controls

Control	Implementation	Security effect
Security headers	helmet()	Adds common HTTP response headers that reduce browser attack surface.
CORS	Configured origin allow-list and credentials option	Rejects browser origins that are not allowed when CORS_ORIGIN is configured.
Login throttling	20 attempts per 15 minutes	Reduces simple online password guessing against /api/auth/login.
Body limit	express.json({ limit: 1mb })	Restricts oversized JSON request bodies.
Request timeouts	Database and frontend fetch timeouts	Avoids indefinitely hanging requests.
Request logging	morgan	Provides operational request traces; production format omits development verbosity.

8.3 Environment and secret handling

Database server, database name, SQL login names, passwords, encryption settings, CORS origins, pool sizes, and timeouts are loaded from environment variables. The source package provides .env.example placeholders rather than real credentials. The active .env file must remain outside source control and must use unique strong passwords.

9. Error handling and session failure behavior

Condition	HTTP result	System behavior
Missing or invalid bearer token	401 Unauthorized	Request stops before the route handler.
Expired session	401 Unauthorized	Session is deactivated, LogoutTime is set, SessionExpired is audited, and the frontend clears local state.
Valid session but insufficient role	403 Forbidden	Route or procedure denies the operation without applying business changes.
Invalid input or business rule failure	400-level response	Stored procedure raises an error and transactional work is rolled back where applicable.
Unknown API route/report key	404 Not Found	No database operation is attempted.
Unexpected server failure	500 Internal Server Error	Production responses omit stack traces; development responses include them.

The frontend treats a 401 response as a session failure, clears the stored token and user context, and redirects to the login page. This client behavior improves usability, but the server-side rejection is the actual security control.

10. Current limitations and improvement priorities

The current implementation contains substantial security controls, but several items should be addressed before production deployment. The priorities below are based on the actual code path and configuration behavior.

Priority	Current condition	Recommended action
Critical	Passwords use unsalted SHA-256.	Move password verification to an adaptive password hash with a unique salt. Migrate existing hashes during successful login or through a controlled reset.
Critical	TLS is not enforced by the Express application; local defaults use DB_ENCRYPT=false and trustServerCertificate=true.	Terminate HTTPS at a trusted reverse proxy or in Node, require encrypted SQL connections, and validate server certificates in production.
High	Bearer tokens may be stored in localStorage, which is readable by injected JavaScript.	Use strong Content Security Policy, eliminate inline script risks, and consider secure HttpOnly SameSite cookies with CSRF protection for production.
High	/api/setup/initial-high-admin is unauthenticated and remains mounted after bootstrap.	Disable or remove the route after initial provisioning, or protect it with a deployment-only secret and network restriction.
High	/api/health is unauthenticated and returns database name, SQL login, and database user.	Return only a minimal health status publicly; restrict detailed diagnostics to trusted operators.
High	CORS allows all origins when CORS_ORIGIN is empty.	Require an explicit production allow-list and fail closed when it is missing.
Medium	Login throttling uses the default process-local store.	Use a shared rate-limit store and combine IP, account identifier, and progressive lockout controls in multi-instance deployments.
Medium	No MFA, password reset, password history, or compromised-password checks are implemented.	Add account recovery and stronger authentication policy for privileged roles.
Medium	Optional report pools reuse app credentials when separate credentials are not configured.	Require distinct SQL credentials in production and verify effective permissions during startup.
Medium	SQL errors may return procedure names, line numbers, and detailed messages; development returns stack traces.	Set NODE_ENV=production and map internal database errors to stable public error codes while preserving detail in server logs.
Low	Initial HighAdmin minimum password length is six while customer signup requires eight.	Apply one centralized password policy to every user-creation procedure.
Low	Session tokens are stored in plaintext in dbo.Sessions.	Store a cryptographic hash of the token and compare hashes, so a database read does not immediately expose usable bearer tokens.

10.1 Recommended production security baseline

1. Replace password hashing and enforce one password policy for all account-creation paths.
2. Enable HTTPS, secure SQL transport, explicit CORS origins, and production error handling.
3. Remove or tightly restrict bootstrap and diagnostic endpoints.
4. Configure separate SQL Server logins for application, reporting, audit, and HighAdmin reporting pools.
5. Add MFA and stronger session controls for Admin and HighAdmin accounts.
6. Add automated tests for unauthenticated access, role denial, branch isolation, ownership isolation, session expiry, and suspended-employee privilege removal.
7. Review audit retention, access to audit views, log monitoring, and incident-response procedures.

Overall assessment

The architecture correctly places the final authorization decisions on the server and in the database, uses current-state effective roles, and separates application identity from SQL Server identity. The largest remaining risks are password hashing, deployment transport/configuration, browser token storage, and publicly mounted bootstrap/diagnostic endpoints.

Appendix A. Key implementation files

Area	File or database object
Express authentication	backend/src/middleware/auth.js
Auth endpoints	backend/src/routes/auth.routes.js
API hardening	backend/src/app.js and backend/src/config/env.js
Database connection pools	backend/src/config/db.js
Typed procedure execution	backend/src/utills/procedure.js
Ownership and branch scope	backend/src/utills/access-scope.js
Browser token handling	frontend/assets/js/core/session.js and api.js
Password hash	dbo.fn_HashPassword
Login	dbo.sp_User_Login
Session validation	dbo.sp_User_ValidateSession
Logout	dbo.sp_User_Logout
Effective-role helper	dbo.fn_UserHasEffectiveRole
Identity/session schema	dbo.Users, dbo.UserRoles, dbo.Roles, dbo.Sessions, dbo.AuditLog
Database roles and users	database/security/Security/01_DatabaseRolesAndUsers.sql
Database grants	database/security/Security/02_DatabasePrivileges.sql
Controlled reports	backend/src/routes/reports.routes.js

Appendix B. Authentication and authorization terminology

Term	Meaning in this project
Authentication	Proving which application user is making the request by validating credentials or an active session token.
Authorization	Deciding what the authenticated user may do based on effective roles, ownership, branch, state, and business rules.
Assigned role	A row in dbo.UserRoles. It may remain stored even when it is not currently usable.
Effective role	A role that is currently valid after checking user, customer, employee, job title, admin flag, and employment state.
Session	A server-side dbo.Sessions row referenced by an opaque bearer token.
Application identity	A person represented by dbo.Users and related business tables.
Database identity	A SQL Server login/user used by the Node.js service or reporting connections.
Scope	The set of records visible to a user: own records, current branch, or all branches.

End of document